

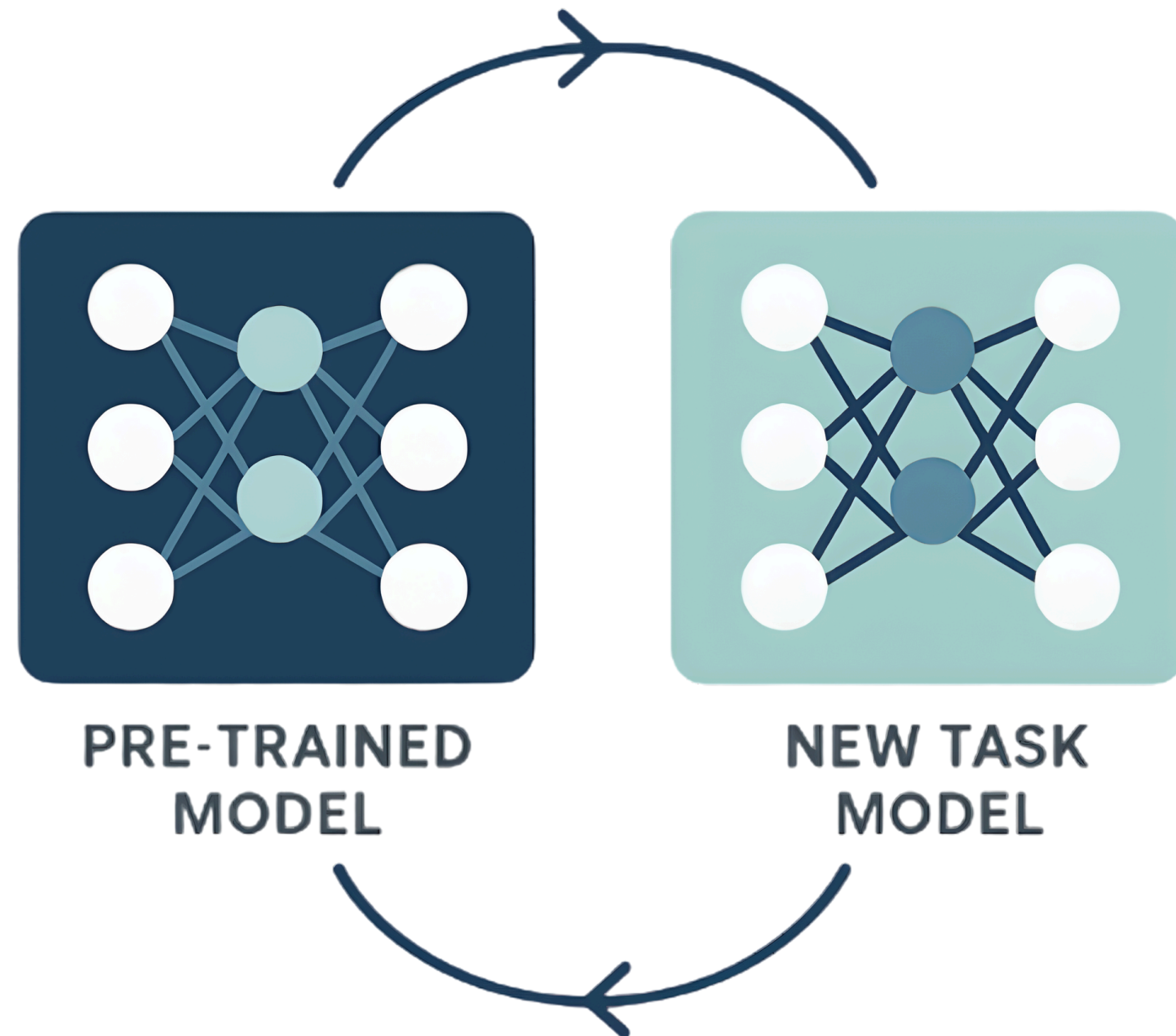
TRANSFER LEARNING

USING EXISTING MODELS TO SOLVE NEW TASKS

PRESENTED BY: BREEHA, ASHBAH & NAMEL



Why Are We Talking About Transfer Learning?

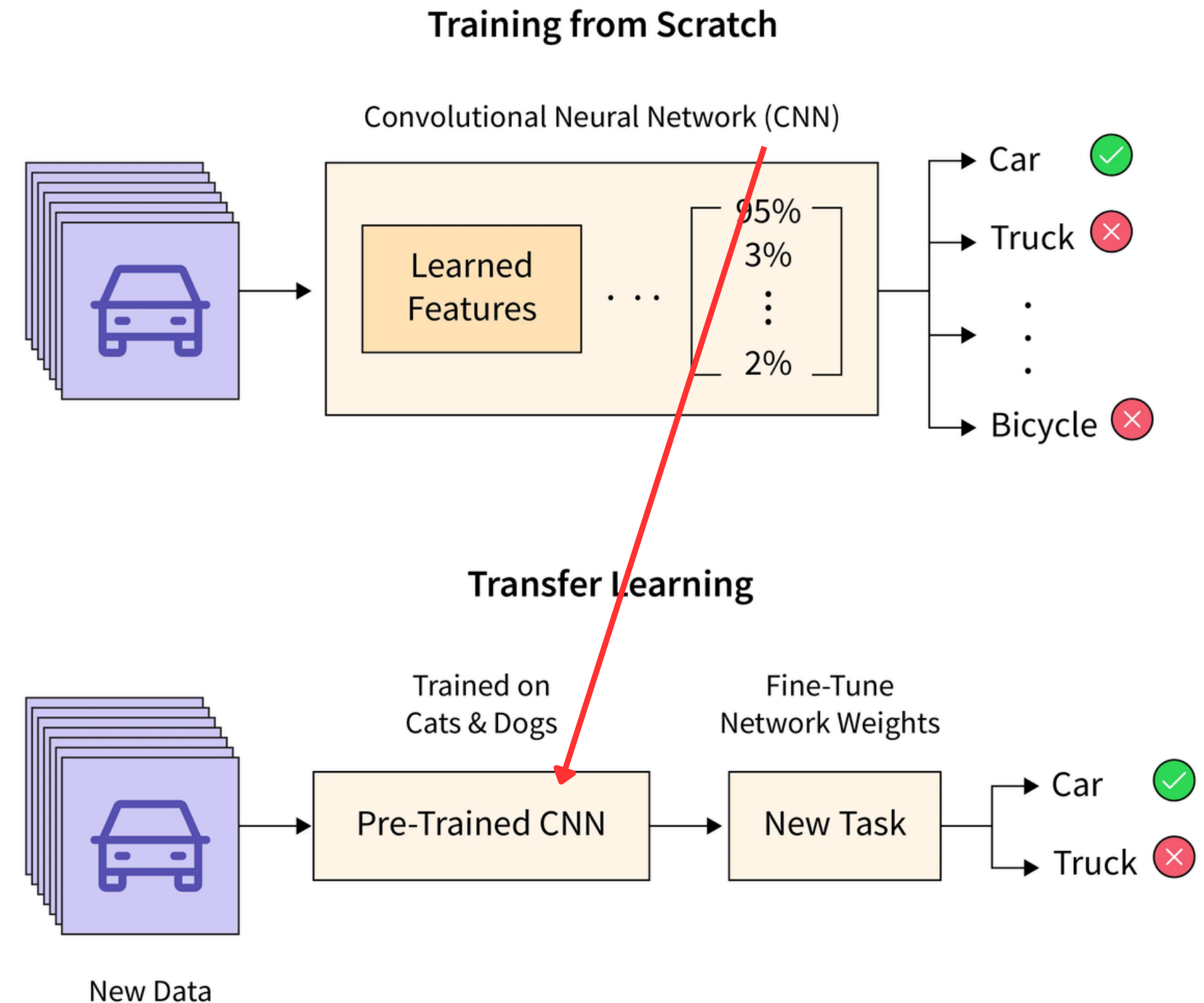


- Training deep learning models from scratch:
 - Needs huge labeled datasets
 - Takes a lot of time and compute resources
 - Example: Training an ImageNet model can take days or weeks
- In real projects:
 - We usually have limited data (e.g., medical images, industrial data)
 - We may not have powerful hardware (hundreds of GPUs)
- We need a cheaper, faster, and practical way to build strong models

This is where Transfer Learning comes in.

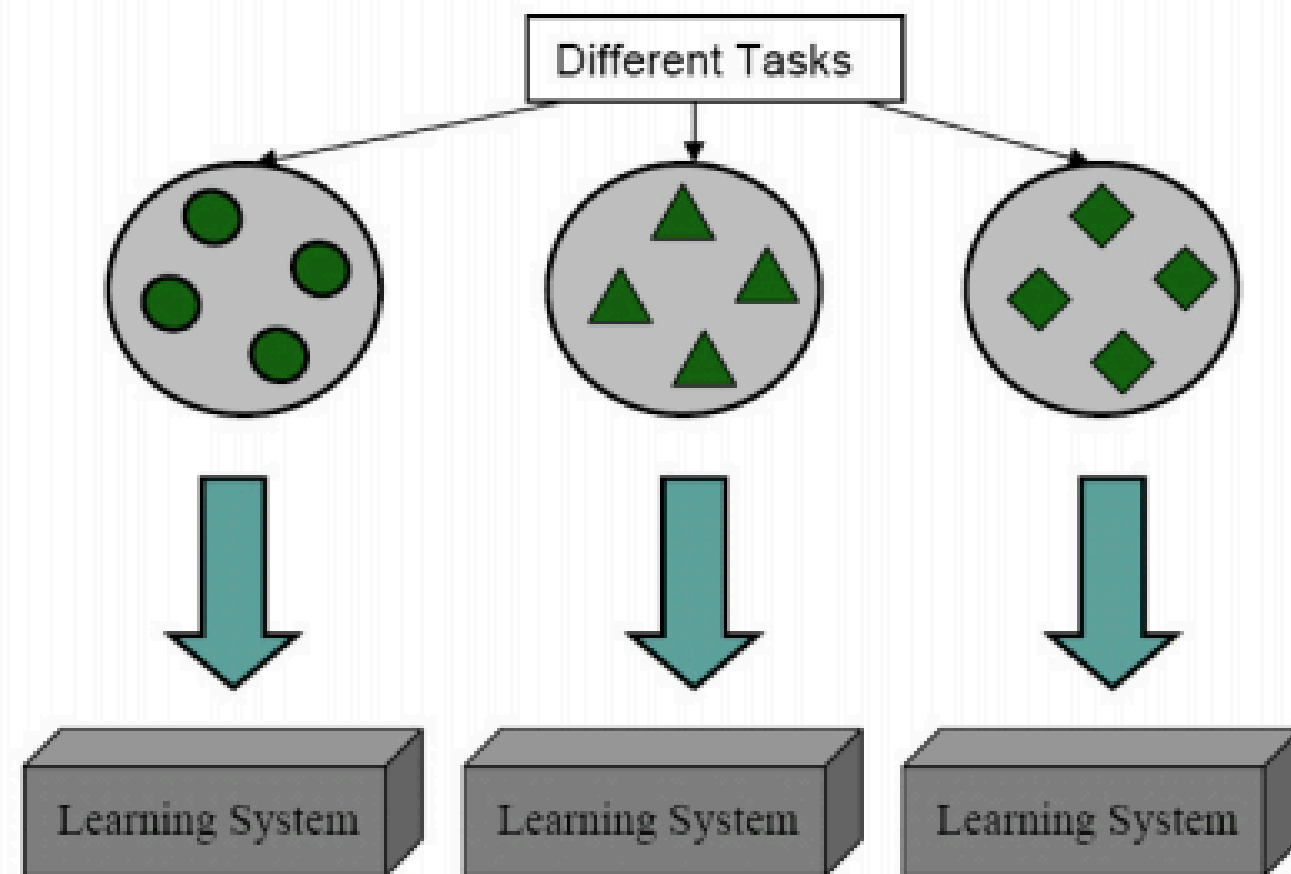
What Is Transfer Learning?

- Transfer Learning is a technique where:
 - A model is first trained on a large, general task
 - Then adapted to a new but related task
- The pretrained model has already learned:
 - Useful patterns and features
 - General knowledge about images, text, or audio
- Instead of starting from zero:
 - We reuse that knowledge and only adjust it a bit



Transfer Learning vs Traditional Machine Learning

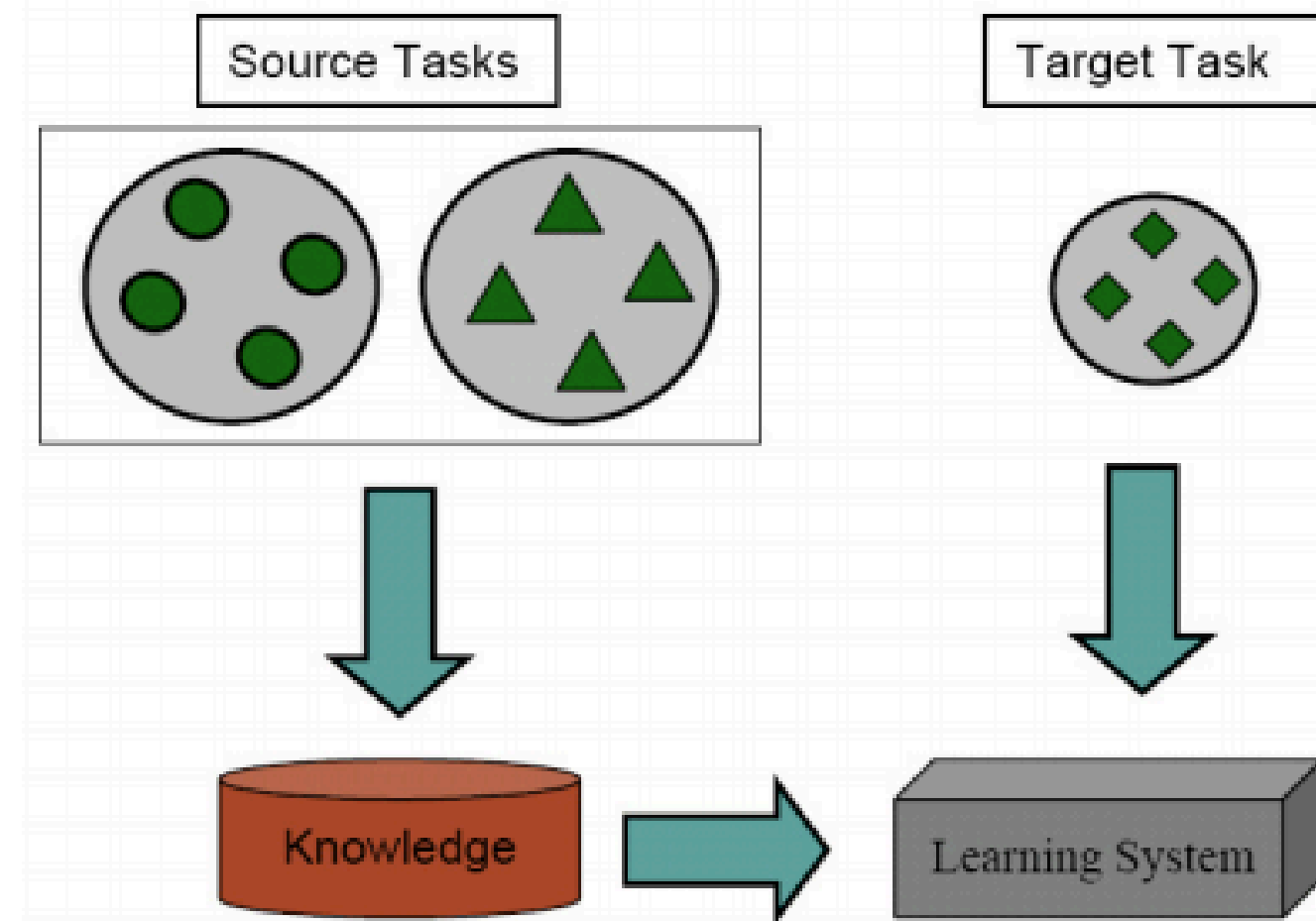
Learning Process of Traditional Machine Learning



(a) Traditional Machine Learning

- It is a wider domain that encompasses different algorithms and techniques for learning from data
- Here the developer needs large volumes of labeled data to train models
- The process is a bit slower and hence its implementation needs more time.
- Can be applied to a wide range of tasks

Learning Process of Transfer Learning



(b) Transfer Learning

- This technology focuses on reusing pre-trained models for related tasks
- It requires fewer labeled examples for training
- It ensures faster results and enhanced performance.
- Limited to tasks that are related to the source task

Why Is Transfer Learning So Popular?

Data scarcity

- In domains like healthcare, robotics, specialized industry, labeled data is rare and expensive

Computational cost

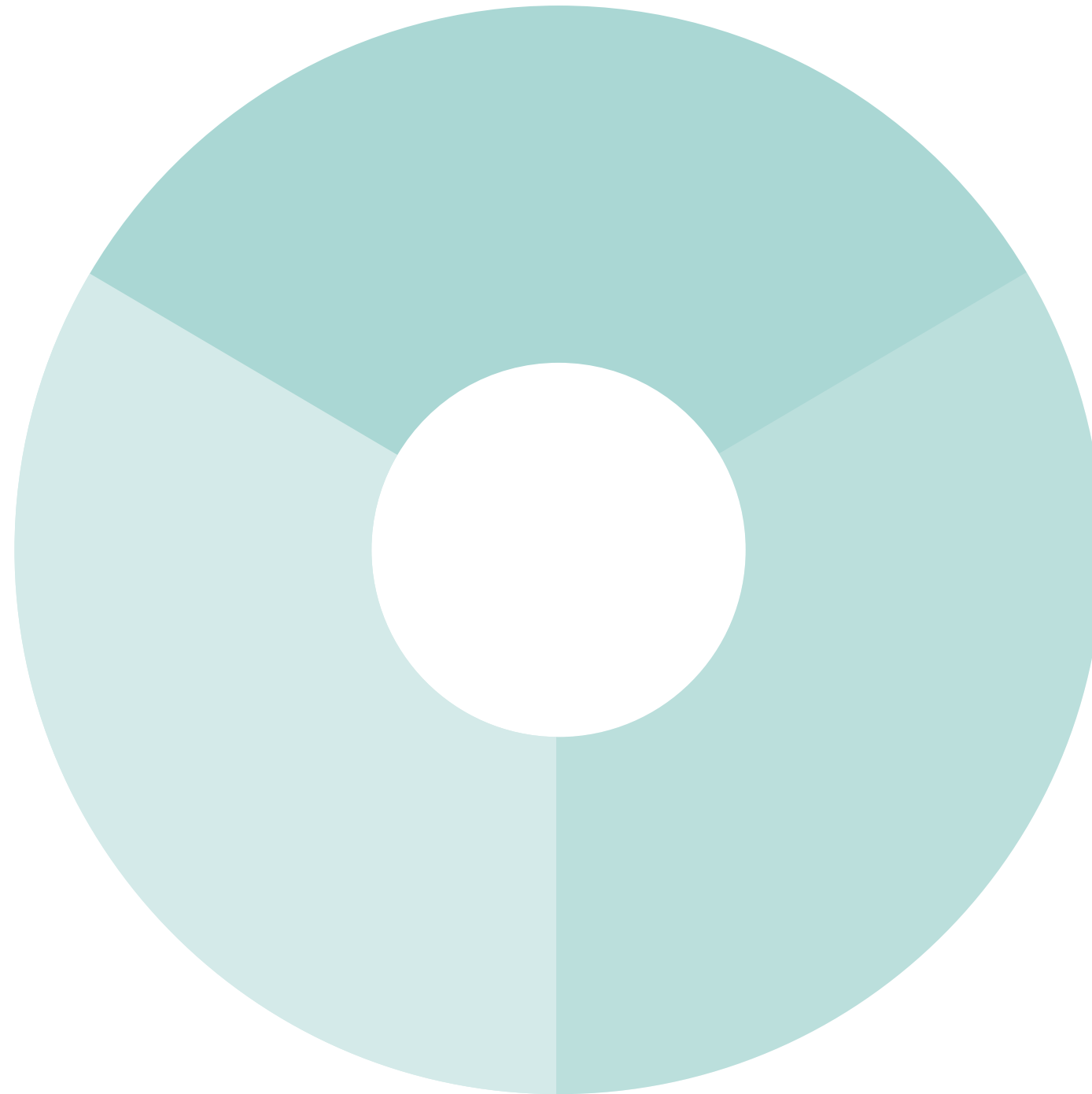
- Training big models from scratch needs powerful GPUs and a lot of time

Time efficiency

- Transfer Learning lets you build good models in hours or days, not weeks

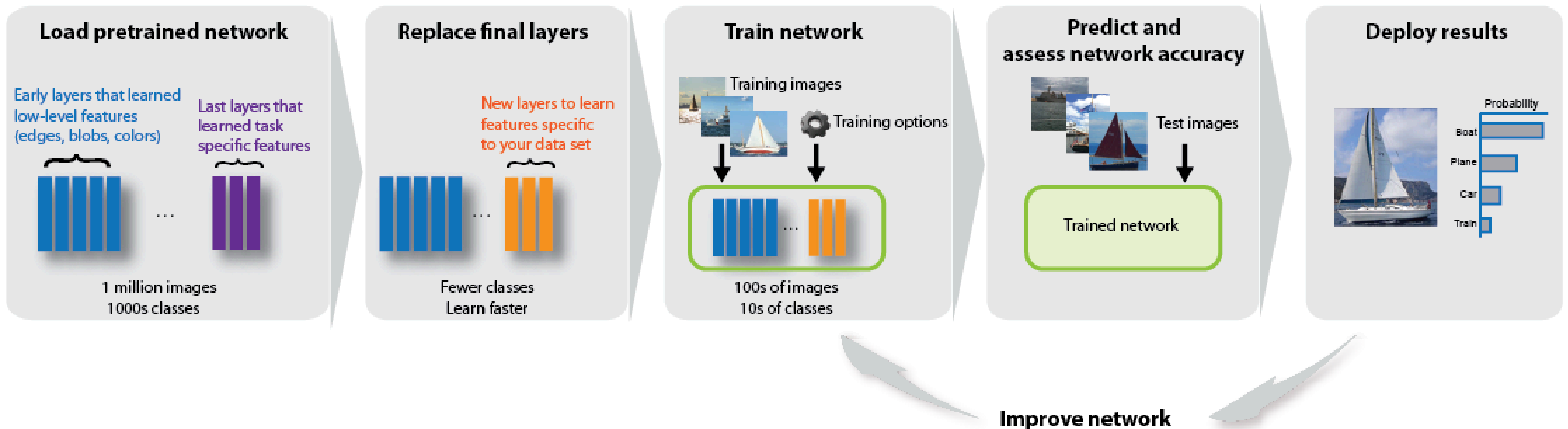
Market impact

- Deep learning market is growing rapidly, and Transfer Learning:
 - Makes AI more accessible
 - Reduces cost and barriers for small teams and companies



High-Level Transfer Learning Workflow

Reuse Pretrained Network



High-Level Transfer Learning Workflow

Step 1 – Choose a pretrained model

- Huge models have already been trained on massive data:
 - ResNet / VGG / MobileNet → trained on ImageNet (millions of images, 1000 classes)
 - BERT / GPT → trained on gigantic text corpora (Wikipedia, books, web text)
 - YOLO → trained on COCO and similar datasets for object detection
- These models have already learned useful patterns:
 - Vision models know what edges, textures, shapes, and common objects look like.
 - Language models know grammar, meaning, and patterns in text.

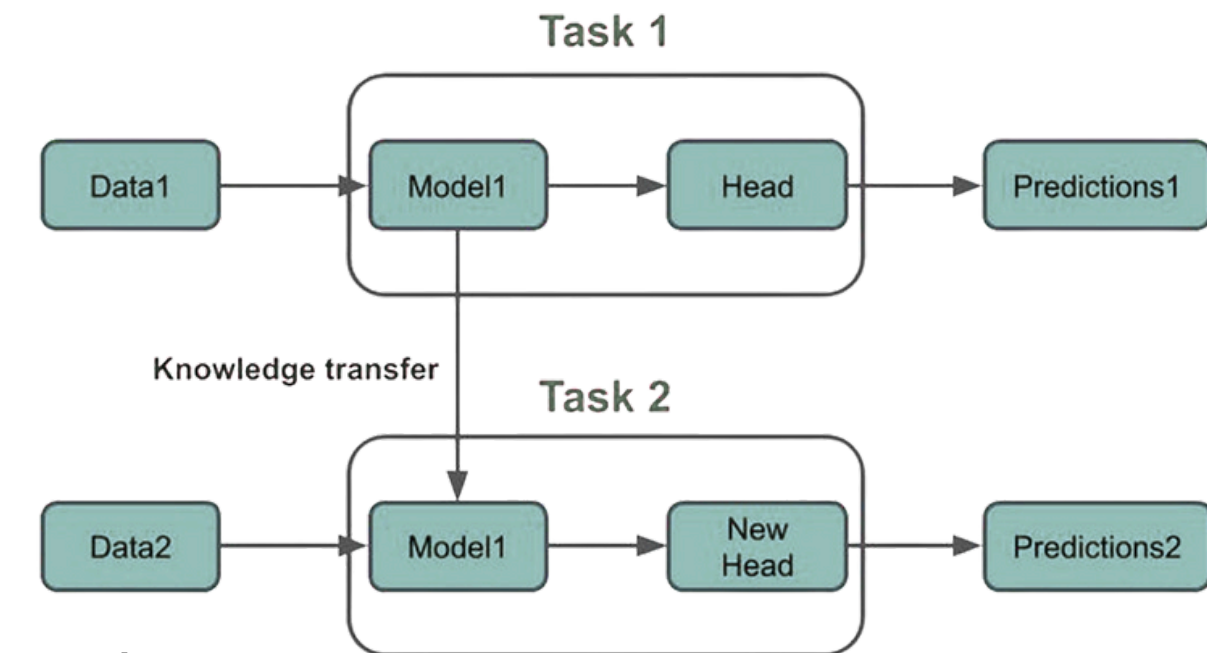
How do you use it?

- You don't train them from scratch.
- You just download the pretrained weights from a library (like PyTorch, TensorFlow, HuggingFace) and load the model.

High-Level Transfer Learning Workflow

Step 2 – Decide what to keep and what to replace

- Most deep models have two parts:
 - Feature extractor (backbone)
 - Early + middle layers
 - Input: raw data (image pixels or tokens)
 - Output: a compact feature vector that represents what's in the input
 - This representation encodes what is in the input in a compact, meaningful way.
 - Classification head (output layer)
 - Last layer(s) on top
 - Input: feature vector from backbone
 - Output: class scores for the original task (e.g., 1000 ImageNet classes)
- For transfer learning:
 - We keep the backbone because it learns general features (edges, shapes, language patterns).
 - We remove and replace the final head because our labels are different.
 - **Why do we replace the head?**
 - The pretrained head is tied to the original labels:
 - If it was trained on 1000 ImageNet classes, its last layer has 1000 outputs while our problem has different label set



High-Level Transfer Learning Workflow

Step 3 – Freeze the feature extractor

Only train the new classifier first

- Freezing means marking the backbone's weights as non-trainable.
- During training:
 - Input → frozen backbone → features (its weights do not change)
 - Features → new classifier → predictions
 - Loss and gradients update only the new classifier's weights

Why do this?

- The pretrained backbone already learns good general features.
- If your new dataset is small, updating all layers can cause overfitting and make it forget that knowledge.
- Freezing:
 - Preserves learned features
 - Makes training faster and cheaper
 - Uses the backbone as a fixed feature encoder.

High-Level Transfer Learning Workflow

Step 4 – Train on your new dataset

Use your own images, text, etc.

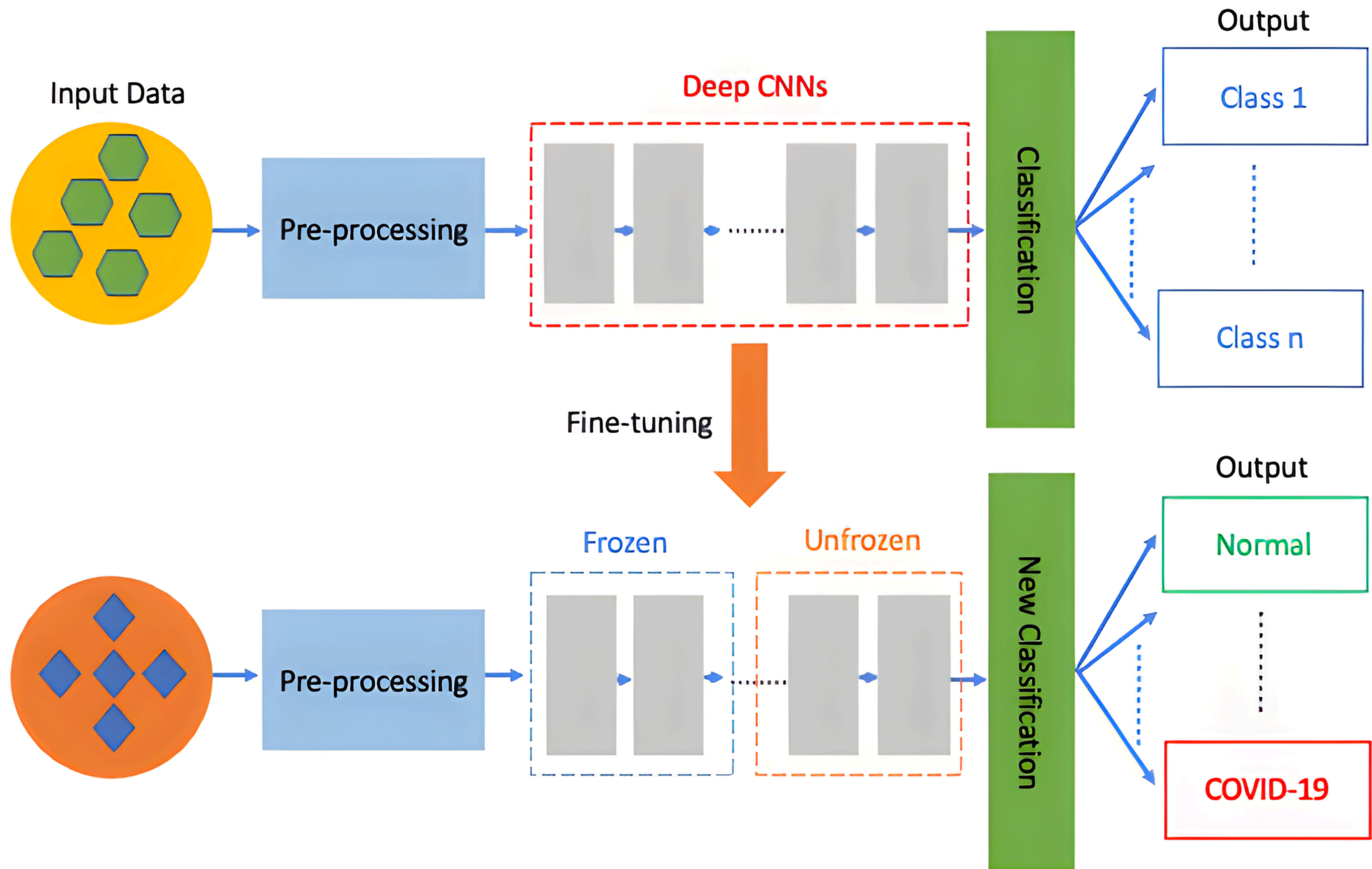
- Now you feed in your labeled data (images, text, etc.).
- For each batch:
 - Input → frozen backbone → feature vector
 - Feature vector → new classifier head → class scores
 - Compare scores with true labels → compute loss
 - Backprop + optimizer → only classifier weights are updated (backbone stays fixed)

Intuition:

- The backbone gives stable, meaningful features; the new head just learns how to map those features to your specific labels.

Step 5 – Fine Tune (Optional)

- After the new classifier works, you can unfreeze some of the deeper backbone layers.
- Now training updates both the classifier head and a few of those deeper layers.
- This lets the model adjust its internal features specifically for your new task.



High-Level Transfer Learning Workflow

Step 6 – Evaluate & deploy

Check accuracy, precision, recall, etc.

Evaluate:

- Use a separate validation/test set the model hasn't seen.
- Run forward passes only, get predictions, compare with true labels.
- Compute metrics like:
 - Accuracy
 - Precision, Recall, F1 (important for imbalanced data)

If results are good enough:

Deploy:

- Save the model (backbone + your classifier head).
- In production:
 - Only forward pass: new input → backbone → head → prediction
 - No training, no weight updates.
- Used in web apps, APIs, or mobile apps to make predictions on new data

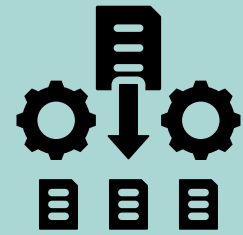
When to Fine-Tune?

Dataset size	Dataset similarity	Recommendation
Large	Very different	Train model B from scratch, initialize weights from model A
Large	Similar	OK to fine-tune (less likely to overfit)
Small	Very different	Train classifier using the earlier layers (later layers won't help much)
Small	Similar	Don't fine-tune (overfitting). Train a linear classifier

Learning Rate in Transfer Learning

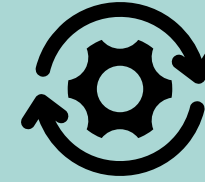
- Training from scratch:
 - Use a normal learning rate (e.g., for your optimizer)
- Fine-tuning pretrained layers:
 - Use a smaller learning rate
 - Why?
 - Pretrained weights are already good
 - We only want to adjust them slightly
 - Large learning rate might destroy useful learned features

Key Ways to Do Transfer Learning



Feature Extraction

- Use the pretrained model as a fixed feature extractor
- You freeze all layers and only train a new classifier on top
- Good when:
 - You have very little data
 - Tasks are reasonably similar



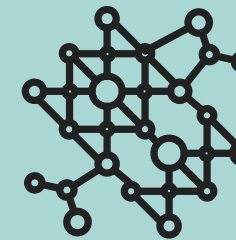
Domain Adaption

- Adapting a model trained in one domain to a new domain
- Example: Photos taken in daylight → adapt to nighttime or different camera type



Fine Tuning

- Start with a pretrained model
- Unfreeze some layers (usually deeper ones)
- Train the model on your new dataset with a small learning rate
- Good when:
 - You have more data
 - Tasks are similar, but you want better accuracy



Multi-task Learning

- One model trained on multiple related tasks at the same time
- Tasks share features and help each other improve



Zero-shot Learning

- Model can handle new classes or tasks it has never seen during training
- Uses high-level descriptions or general knowledge learned earlier

Transfer Learning Applications

Convolutional Neural Networks (CNNs)

CNNs trained on large datasets can be adapted for specific, more specialised tasks in image classification and object detection tasks.

01

Recurrent Neural Networks (RNNs)

In NLP, RNNs trained on vast text corpora can be fine-tuned for tasks like sentiment analysis, chatbots, and translation.

02

Robotics

Transfer Learning is used in robotics to help machines adapt to new environments and tasks, such as navigation and object manipulation, with less data.

03

Healthcare

Transfer Learning allows providers to adapt pre-trained models to diagnose diseases from medical images like X-rays and CT scans, significantly improving diagnosis accuracy with limited data.

04

Cloud Services for Transfer Learning

Why cloud services?

- Deep models like Inception-V3 or ResNet require millions of parameters and huge datasets.
- Cloud platforms allow you to reuse these models and retrain only the final layers on very small datasets (e.g., 19 images per class of galaxies).
- Few modern cloud services that rely on TL:
 - **The IBM Watson Visual Recognition tool**
 - Drag-and-drop web interface, no ML expertise needed
 - Upload zipped class folders, it handles transfer learning internally
 - Shows class probabilities; ~72% accuracy on galaxies
 - Sometimes refuses to classify very unclear images
 - **Microsoft Azure Custom Vision**
 - GUI like Watson: upload images → click Train
 - Fast training; easy to iterate by adding/removing images
 - Best result in this study: ~75% accuracy
 - Still confuses similar classes (barred spiral vs spiral)
 - **Google TensorFlow – “TensorFlow for Poets”**
 - Code-based; needs basic Python/TF
 - Retrains Inception-V3 locally by replacing top layers
 - Only final layer trained → fast; ~67% accuracy
 - More control, not a black box

ADVANTAGES

- **Less data needed:** Works well even when you only have a small labeled dataset.
- **Faster training:** You reuse pretrained weights and only train/fine-tune the last layers.
- **Lower compute cost:** No need to train huge models from scratch on big GPU clusters.
- **Better performance:** Pretrained models learned from large, diverse datasets → often higher accuracy and better generalization.
- **Easy to use via services:** Cloud tools (Azure Custom Vision, IBM Watson, SageMaker, TF Hub, etc.) hide most of the complexity.

DISADVANTAGES

- **Domain mismatch / negative transfer:** If source and target tasks are very different, performance can even get worse.
- **Limited control in cloud services:** Black-box APIs (Watson, Azure) don't expose how the model is really working.
- **Large model sizes:** Pretrained models can be heavy and hard to deploy on edge / mobile devices.
- **Hyperparameter sensitivity:** Fine-tuning (e.g., in SageMaker / TensorFlow) needs careful tuning of learning rate, layers to unfreeze, etc.
- **Bias and data dependence:** Any bias in the original pretraining data can be carried over into your downstream task.

**Thank you
very much!**